

Replication Workplan – Baqaee & Farhi (2019) Beyond Hulten’s Theorem

July 2026

1 Replication Workplan – Baqaee & Farhi (2019)

1.1 Scope

This workplan describes a from-scratch, clean-room replication of the computational results in Baqaee & Farhi (2019), “The Macroeconomic Impact of Microeconomic Shocks: Beyond Hulten’s Theorem” (*Econometrica*, 87(4), 1155–1203).

The original MATLAB code is located in `Replication Files/GDP Simulatin -- 88 Sector/`. This replication will use Julia for the core model and Jupyter notebooks for documentation, visualization, and analysis. The goal is a transparent, well-documented, reproducible codebase that can serve as the computational foundation for the revised manuscript.

1.2 Original Code Inventory

1.2.1 Core solver functions

File	Lines	Purpose
<code>Simulation.m</code>	20	Core equilibrium solver: 2N system for prices p and quantities y
<code>Simulation_Derivs.m</code>	34	Solver with analytical Jacobian (for Knitro/fmincon)
<code>Simulation_Derivs_realloc.m</code>	23	Solver variant with mobile labor ($w = 1$, no sector-specific wages)

1.2.2 Pipeline scripts

File	Lines	Purpose
<code>getData.m</code>	83	Data loading from <code>us80dbasedata.mat</code> to <code>simulationData.mat</code>
<code>GDP_Simulation_88sectorKLEMS_314</code>	314	Main simulation: random TFP shocks, GDP computation, robustness
<code>elasticity_gradient.m</code>	200	Elasticity sensitivity sweep across epsilon, theta, sigma
<code>GDP_Simulation_88sectorKLEMS_188robustness.m</code>	188	Robustness checks (Monte Carlo, 50k simulations)
<code>GDP_Simulation_88sectorKLEMS_179reallocation.m</code>	179	Simulation with mobile labor (reallocation)
<code>Second_Order_Simulation.m</code>	171	Second-order welfare effects beyond Hulten’s theorem
<code>GDP_realloc_function.m</code>	–	Labor reallocation helper
<code>lambda_realloc_function.m</code>	–	Domar weight reallocation helper

1.2.3 Data files

File	Description
us80dbasedata.mat	Jorgenson 88-sector US data (1960–2005), raw format
simulationData.mat	Processed data (Omega, alpha, beta, domar_weights, L)
stfp.mat / stfp.csv	Sectoral TFP growth rates (Carvalho-Gabaix)
NOMINAL_USE_1993.DAT–NOMINAL_USE_2008.DAT	Use tables
allUSdata.mat	Alternative data format
BFdata.csv	CSV export of processed data
qgdp.xls / qgdp1.mat	Quarterly GDP data
jacobian.mat	Pre-computed Jacobian

1.2.4 Supporting files

File	Purpose
industries_data_privatesector_1960_2008.m	TFP extraction (Carvalho-Gabaix method)
BLS_to_Jorgenson.m	BLS to Jorgenson sector mapping
mvnrnd.m	Multivariate normal random draws (MATLAB stats)
Knitro_Options.opt	Knitro solver configuration
matlab2tikz.m	TikZ export (6805 lines, utility)
nhist.m	Histogram utility (2149 lines)

1.2.5 Additional replication folders

Folder	Purpose
Growth Accounting_Klems/	Section 6.3 growth accounting
Stuck Intermediates and Adjustment Costs/	Online Appendix 2 (adjustment costs)
Robustness/	Online Appendix 3 robustness tables

1.3 The Model to Replicate

1.3.1 Equilibrium system

The B&F (2019) model is a one-factor (labor) CES economy with N sectors. The equilibrium is defined by a $2N$ system of equations in prices p (N) and quantities y (N).

Price equations (zero profit):

$$p_u = (A_u^{(e-1)} * (\alpha_u * w_u^{(1-e)} + (1-\alpha_u) * q_u^{(1-e)}))^{1/(1-e)}$$

Market clearing (goods):

$$y_u = \sum_s \omega_{su} * p_s^e * A_s^{(e-1)} * q_s^{(\theta-e)} * (1-\alpha_s) * y_s * p_u^{(-\theta)} + c_u$$

Supporting equations:

$$q_u = (\sum_s \omega_{us} * p_s^{(1-\theta)})^{1/(1-\theta)}$$

$$w_u = p_u * A_u^{((e-1)/e)} * \alpha_u^{(1/e)} * y_u^{(1/e)} * l_u^{(-1/e)}$$

$$c_u = D * (p_u / \text{CPI})^{(-\sigma)} * \omega_{0u} * \sum_s w_s * l_s$$

$$\text{CPI} = (\sum_u \omega_{0u} * p_u^{(\theta-1)})^{1/(\theta-1)}$$

1.3.2 Key parameters

- ϵ (epsilon): elasticity between intermediates and labor (default 0.5)
- θ (theta): elasticity between intermediate goods (default 0.001, near-Leontief)
- σ (sigma): elasticity of consumption substitution (default 0.9)
- α (alpha): factor share (from IO table)

- Ω (Ω): cost share matrix (from IO table)
- β (β): consumption shares (from IO table)
- A ($\mathcal{A}$): productivity parameter (calibrated to 1 in baseline)
- L : labor allocation (Domar weights times factor share in baseline)

1.3.3 Data processing pipeline

1. Load `us80dbasedata.mat` (Jorgenson 88-sector, 1960–2005)
2. Reshape into sector x year matrices: `grossy`, `capital`, `labor`, `vadd`
3. Remove government sectors (indices 60, 80–88) and zero-sales sectors (8, 62)
4. Select base year (1980 in main script, 1982 in `getData.m`)
5. Extract IO matrix, normalize to cost shares to obtain Ω
6. Compute factor shares: `alpha = vadd / grossy`
7. Compute consumption shares: `beta = grossy' * (I - diag(1-alpha) * Omega)`, normalize
8. Compute Domar weights: `lambda = beta' * inv(I - diag(1-alpha) * Omega)`
9. Compute baseline labor allocation: `L = lambda .* alpha`
10. Compute TFP data from Carvalho-Gabaix method to obtain `stfp`
11. Save processed data to `simulationData.mat`

1.3.4 Solver

Original: Knitro (commercial) with `fmincon` fallback. The solver minimizes $\|Out\|$ where `Out` is the $2N$ residual vector from the equilibrium equations. The analytical Jacobian (`Simulation_Derivs.m`) is provided but the original code also works with finite differences.

For the Julia replication: use `NonlinearSolve.jl` (already proven to work in the `BeyondHulten` package).

1.4 Replication Structure

```
bf_replication/
  REPLICATION_WORKPLAN.md      -- this file
  Project.toml                 -- Julia project (to be created)
  data/                        -- symbolic links to original data
    us80dbasedata.mat
    simulationData.mat
    stfp.csv
    BFdata.csv
  src/
    BFReplication.jl          -- module (to be created)
    data_loader.jl            -- data loading + processing
    model.jl                  -- equilibrium system (problem function)
    solver.jl                  -- NonlinearSolve wrapper
    shocks.jl                  -- TFP shock generation
    gdp.jl                     -- GDP computation (real, nominal)
    domar_weights.jl           -- Domar weight extraction
    second_order.jl           -- second-order welfare effects
  notebooks/
    01_data_loading.ipynb     -- data pipeline documentation
    02_benchmark_replication.ipynb -- baseline equilibrium + GDP
    03_elasticity_gradient.ipynb -- elasticity sensitivity sweep
    04_robustness.ipynb        -- Monte Carlo robustness
    05_reallocation.ipynb      -- mobile labor variant
    06_second_order.ipynb      -- second-order effects
  tests/
    runtests.jl                -- unit tests
```

1.5 Phased Workplan

1.5.1 Phase R1: Data Pipeline

Goal: Load and process the Jorgenson data in Julia, reproduce the processed data matrices.

Tasks:

- Create Julia project with dependencies: `MAT.jl`, `DataFrames.jl`, `CSV.jl`, `LinearAlgebra.jl`, `NonlinearSolve.jl`
- Implement `data_loader.jl`:
- Load `us80dbasedata.mat` using `MAT.jl`
- Reshape into sector x year matrices (88 sectors, 46 years)
- Remove government and zero-sales sectors to get 78 sectors
- Extract IO matrix for base year
- Compute `Omega`, `alpha`, `beta`, `lambda` (Domar weights), `L`
- Verify against `simulationData.mat` (load and compare)
- Load `stfp.csv` (sectoral TFP) and compute covariance matrix
- Document in `01_data_loading.ipynb`

Verification: Processed `Omega`, `alpha`, `beta`, `lambda`, `L` must match `simulationData.mat` to machine precision.

Estimated effort: 3–5 days

1.5.2 Phase R2: Core Equilibrium Solver

Goal: Implement and verify the equilibrium solver.

Tasks:

- Implement `model.jl`:
- Port `Simulation.m` to Julia `problem()` function
- The 2N residual vector (N price equations + N market clearing)
- All supporting equations (`q`, `w`, `c`, `CPI`)
- Implement `solver.jl`:
- Wrap `NonlinearSolve.jl` (already proven in `BeyondHulten`)
- Use baseline prices = 1, quantities = `lambda` as initial guess
- Verify convergence for default elasticities ($\varepsilon = 0.5, \theta = 0.001, \sigma = 0.9$)
- Implement `gdp.jl`:
- Real GDP via Laspeyres index
- Nominal GDP from supply side ($\sum w_s l_s$)
- CPI computation
- Port `Simulation_Derivs.m` to analytical Jacobian (optional, for verification)
- Document in `02_benchmark_replication.ipynb`

Verification: Baseline (no shock) solution must have prices approximately 1, quantities approximately `lambda`, `GDP` = 1.

Estimated effort: 3–5 days

1.5.3 Phase R3: Shock Simulation

Goal: Reproduce the random TFP shock simulations from B&F Section 6.1.

Tasks:

- Implement `shocks.jl`:
- Multivariate normal draws from TFP covariance matrix (`stfp` data)
- 4-year cumulative shocks (matching the original methodology)
- Supply shocks (`A` parameter changes) and demand shocks
- Reproduce `GDP_Simulation_88sectorKLEMS.m`:
- Run 50,000 Monte Carlo simulations
- Record GDP, sectoral prices, sectoral quantities for each
- Compute distributions (histograms, moments)
- Document in `03_elasticity_gradient.ipynb` (elasticity sweep) and `04_robustness.ipynb` (Monte Carlo)

Verification: GDP distribution moments should match B&F Figure 6 and the robustness tables in Online Appendix 3.

Estimated effort: 5–7 days

1.5.4 Phase R4: Elasticity Gradient

Goal: Reproduce the elasticity sensitivity analysis.

Tasks:

- Port `elasticity_gradient.m`:
- Sweep each elasticity across `linspace(0.015, 0.99)` while holding others constant
- For each combination: solve equilibrium, record GDP and sectoral outcomes
- Compare with Cobb-Douglas benchmark
- Reproduce B&F Figure 7 (elasticity gradient plots)
- Document in `03_elasticity_gradient.ipynb`

Verification: GDP-vs-elasticity curves should match the original figures qualitatively (monotonic in each elasticity).

Estimated effort: 2–3 days

1.5.5 Phase R5: Mobile Labor Variant

Goal: Reproduce the labor reallocation variant.

Tasks:

- Port `Simulation_Derivs_realloc.m`:
- Set $w = 1$ (economy-wide wage, mobile labor)
- Solve only the N price equations (quantities determined by labor allocation)
- This is the “mobile labor” / “reallocation” case
- Reproduce `GDP_Simulation_88sectorKLEMS_reallocation.m` pipeline
- Document in `05_reallocation.ipynb`

Verification: Mobile labor results should show higher GDP (less constrained) than fixed labor, matching B&F’s reallocation figures.

Estimated effort: 2–3 days

1.5.6 Phase R6: Second-Order Effects

Goal: Reproduce the second-order welfare accounting beyond Hulten’s theorem.

Tasks:

- Port `Second_Order_Simulation.m`:
- Compute first-order GDP change (Hulten: $\Delta \ln GDP = \sum_s \lambda_s \Delta \ln \mathcal{A}_s$)
- Compute second-order terms (involving derivatives of Domar weights)
- Compare first vs second order for various shock sizes
- Document in `06_second_order.ipynb`

Verification: Second-order terms should be small for small shocks and grow nonlinearly for large shocks, matching B&F Figure 4.

Estimated effort: 4–6 days

1.6 Dependencies Between Phases

```
R1 (data) -> R2 (solver) -> R3 (shocks) -> R4 (elasticity gradient)
                                     \-> R5 (reallocation)
                                     \-> R6 (second order)
```

R2 depends on R1. R3–R6 all depend on R2 but are independent of each other and can be parallelized.

1.7 Mapping to the Existing BeyondHulten Package

The existing `BeyondHulten` Julia package (`src/`) already implements much of the model infrastructure. Where possible, the replication should reuse or adapt code from `BeyondHulten`:

BeyondHulten file	Replication equivalent	Reuse?
<code>interface.jl</code> (<code>generate_data</code>)	<code>data_loader.jl</code>	Adapt: B&F US data vs German IO table
<code>ces.jl</code> (<code>problem</code>)	<code>model.jl</code>	Adapt: same equations, different data
<code>ces.jl</code> (<code>solve</code>)	<code>solver.jl</code>	Direct reuse of <code>NonlinearSolve</code> approach
<code>impulses.jl</code> (<code>elasticity_gradient</code>)	<code>notebooks/03</code>	Adapt: US data, same sweep logic
<code>solution.jl</code>	<code>gdp.jl</code>	Adapt: same GDP computation
<code>util.jl</code> (<code>cpi</code>)	<code>gdp.jl</code>	Direct reuse

The replication is **separate** from `BeyondHulten` to keep the clean-room approach. Cross-referencing is encouraged but the replication code should be self-contained.

1.8 Total Estimated Effort

Phase	Days	Cumulative
R1: Data pipeline	3–5	5
R2: Core solver	3–5	10
R3: Shock simulation	5–7	17
R4: Elasticity gradient	2–3	20
R5: Mobile labor	2–3	23
R6: Second-order effects	4–6	29
Total	19–29 days	~4–6 weeks

This can be compressed if `BeyondHulten` code is adapted rather than written from scratch (potentially 2–3 weeks).